A

PROJECT REPORT

ON

CashFlow-Guardian: AI-Powered Financial Management System for Business

Submitted to

G H Raisoni University, Amravati.
School of Science

In partial fulfillment for the requirement of the degree of
Bachelor of Computer Application
(BCA III Year, Semester V)

Submitted by
Jeevan S Kaware
(BCA III Year, Semester V)

Guided by
Dr. Bhavana Karmore
Head of the Department and Assistant Professor, School of Science, GHRU, Amravati

G H Raisoni University, Amravati
Department of Computer Applications
School of Science
2026-27

G H Raisoni University, Amravati
Department of Computer Applications
School of Science
2026-27

CERTIFICATE

This is to certify that the project titled ***"CashFlow-Guardian: AI-Powered FinancialManagement System for Business"*** *has been submitted by* ***Jeevan Kaware*** *in partialfulfillment for the requirement of the degree of* ***Bachelor of Computer Application BCA III Year, Semester V.*** *This is an original work carried out by him/her/themunder the guidance of* ***Dr. Bhavana Karmore*** *and she/he has undergone therequisite hours of project work as specified by* ***G H Raisoni University, Amravati.***

Dr. Bhavana Karmore
Project Guide

Dr. Bhavana Karmore
Head of the Department

Dr./Mr.……
External Examiner

Declaration

I hereby certify that the work which is being presented in the project entitled " CashFlow-Guardian: AI-Powered Financial Management System for Business "submitted in the DEPARTMENT OF COMPUTER APPLICATIONS AT G H RAISONI UNIVERSITY, AMRAVATI. is an authentic record of my own work carried under the supervision of Dr. Bhavana Karmore – Head of Department and Assistant Professor of BCA. This study has not been submitted to any other institution or university for the award of any other degree.

Yours Faithfully
Jeevan Kaware
ENROLLMENT NO :-
GHRUA24011140080
BCA III, Sem V

Date:

G H Raisoni University, Amravati
Department of Computer Application
School Of Science
2026-27

ACKNOWLEDGEMENT

I would like to express my profound gratitude to **Dr. Bhavana Karmore**, Head of Department of BCA and **Dr. Shailesh Thakare**, Dean of SCHOOL OF SCIENCE , G H RAISONI UNIVERSITY for their contributions to the completion of my project titled "CashFlow-Guardian: AI-Powered Financial Management System for Business."

I would like to express my special thanks to project guide **Dr. Bhavana Karmore** for her time and efforts he/she provided throughout the year. Your useful advice and suggestions were really helpful to me during the project's completion. In this aspect, I am eternally grateful to you.

I would like to acknowledgement that this project was completed entirely by me and not by someone else.

Jeevan Kaware
BCA III, Sem V
GHRU, Amravati.

GH RAISONI UNIVERSITY, AMRAVATI
GH Raisoni Nagar, Anjangaon Bari Road, Amravati-444701
School of Science
Department of Computer Applications
Name of Program: Bachelor of Computer Applications | Sem V
Session: 2026-27 (WINTER 2026)

Title of Project :- CashFlow-Guardian: AI-Powered Financial Management System for Business

Abstract :

CashFlow-Guardian is an AI-powered financial management system designed to help businesses manage their financial activities efficiently in one centralized platform. The system provides a structured way to record and monitor income, expenses, budgets, invoices, payments, and cash flow. It aims to improve financial visibility, reduce manual work, and help users maintain accurate financial records for better business management.

The system helps users track business income and expenses, manage cash accounts, maintain customer records, generate invoices, and monitor payment statuses. It provides financial summaries that help users understand their current cash position, outstanding receivables, upcoming expenses, and overdue payments. By organizing financial information in one place, the system makes it easier to review transactions and understand the overall financial condition of the business.

CashFlow-Guardian also focuses on cash-flow forecasting by using recorded financial transactions, outstanding invoices, and upcoming expenses to estimate future cash availability. The system can identify potential cash shortages and provide alerts when payments are overdue or financial obligations require attention. These estimates help users plan expenses, monitor receivables, and make informed financial decisions.

The project is developed using Java and Spring Boot for backend processing, React.js for the user interface, and PostgreSQL for storing financial data. Spring Security is used to support authentication and authorization, while Spring AI integrates AI-assisted capabilities. Tools such as Git, GitHub, and Postman support version control and API testing. Overall, CashFlow-Guardian aims to provide a reliable, organized, and intelligent approach to business financial management.

INDEX

CHAPTER NO.	TITLE	PAGE NO.
1	**Introduction to Project** 1.1 Introduction of proposed project 1.2 Problem Statement 1.3 Objectives 1.4 Scope and Limitations	7
2	**Literature Review** 2.1 Overview of Existing Systems 2.2 Analysis of System 1 2.3 Analysis of System 2 2.4 Analysis of System 3 2.5 Summary and Gap Analysis	11
3	**Methodology** 3.1 System Analysis 3.2 System Design	18
4	**Proposed System** 4.1 Working of Proposed System 4.2 System Requirements ● Hardware Requirement ● Software Requirement.	23
5	**Conclusion**	30
6	**References**	33

CHAPTER 1: INTRODUCTION OF PROJECT

1.1 Introduction of proposed project

CashFlow-Guardian: AI-Powered Financial Management System for Business is a web-based application designed to simplify and organize business financial activities. The system provides a centralized platform to manage income, expenses, cash accounts, customers, invoices, payments, and financial records. It helps users monitor their current cash position, track outstanding receivables, identify overdue payments, and review upcoming expenses. The system aims to improve financial visibility, reduce manual effort, and support efficient business financial management.

Traditional financial management methods often rely on spreadsheets, manual calculations, or separate tools for maintaining financial records. These methods can lead to calculation errors, difficulty tracking unpaid invoices, poor expense monitoring, and delays in understanding the overall cash position. The proposed system addresses these challenges by organizing financial information in a centralized platform. It helps users monitor transactions, manage invoices and payments, review financial summaries, and receive notifications about overdue payments and potential cash-flow problems. Secure authentication and authorization help protect financial information from unauthorized access.

The system is developed using React.js for the frontend, Java and Spring Boot for backend processing, and PostgreSQL for database management. It also uses Spring Security for authentication and authorization and Spring AI with Google Gemini to support AI-assisted financial insights. The main objective of this project is to improve financial record management, monitor cash flow, track receivables and expenses, and support better financial decision-making. In the future, the system can be enhanced with additional forecasting techniques, advanced financial analytics, and improved automated collection recommendations to provide more comprehensive financial management capabilities.

1.2 Problem Statement

Managing business finances becomes difficult when income, expenses, invoices, payments, and outstanding amounts are recorded manually or maintained in separate systems. Such methods can result in inaccurate financial records, missed payment follow-ups, difficulty tracking overdue invoices, and delays in preparing financial summaries. Users may also face challenges in estimating future cash availability, identifying potential cash shortages, monitoring upcoming expenses, and understanding customer payment behavior. The absence of a centralized financial management platform can make it difficult to obtain a clear view of the business's financial position and make timely decisions.

Therefore, there is a need to develop a centralized, web-based financial management system that integrates income and expense tracking, invoice management, payment monitoring, cash-flow forecasting, and AI-assisted financial insights into a single platform. The proposed CashFlow-Guardian system aims to reduce manual effort, organize financial data, provide timely notifications, and help users identify potential financial risks. By presenting relevant financial summaries and estimates based on recorded data, the system supports better planning, improved financial visibility, and informed business decisions.

Using React.js, TypeScript, Tailwind CSS, Java Spring Boot, and PostgreSQL, CashFlow-Guardian provides a centralized financial management solution with interactive dashboards, graphical financial reports, cash-flow forecasting, and AI-assisted insights to support efficient business operations and better financial decisionmaking.

1.3 Objectives of the Project

- Financial Record Management: To maintain centralized financial records for business activities.
- Income and Expense Tracking: To record income, expenses, and financial transactions accurately.
- Invoice and Payment Management: To generate invoices and monitor paid, pending, and overdue payments.
- Cash-Flow Forecasting: To estimate future cash availability using financial records and upcoming obligations.

- AI-Assisted Financial Insights: To provide useful financial insights and suggestions using Artificial Intelligence.
- Financial Alerts: To notify users about overdue payments and potential cash-flow problems.
- Customer Payment Analysis: To analyze customer payment behavior and identify payment delays.
- Financial Reporting: To provide financial summaries, dashboards, graphs, and transaction reports.
- Data Security: To protect financial information through authentication and role-based access control.
- Improve Decision-Making: To help users understand their financial position and make informed business decisions.

1.4 Scope of the Project

The scope of this project includes the development of the following core modules:

- User Management: Provides secure login and access control to protect business financial information.
- Business Management: Enables users to manage business-related financial information through a centralized platform.
- Customer Management: Maintains customer details and organizes customer-related financial records.
- Cash Account Management: Records and monitors cash balances to help users understand their available financial resources.
- Income and Expense Management: Enables users to record expenses, monitor financial transactions, and track financial obligations.
- Invoice Management: Supports invoice creation, invoice tracking, and monitoring of outstanding and overdue amounts.
- Payment Management: Maintains payment records and helps track collected amounts and pending payments.
- Customer Payment Behavior: Analyzes available payment history to identify payment patterns and delays.

- Cash-Flow Forecasting: Estimates future cash positions using current balances, outstanding receivables, and upcoming expenses.
- Notifications and Reminders: Generates notifications for upcoming invoice due dates, overdue payments, expenses, and potential cash-flow risks.
- AI-Assisted Financial Analysis: Uses Spring AI and Google Gemini to support financial insights and recommendations based on available information.
- Dashboard and Reports: Displays financial summaries, graphical representations, transaction information, and cash-flow indicators to support business monitoring.
- Security and Data Management: Uses Spring Security and PostgreSQL to protect access and maintain organized financial records.

1.5 Limitations of the Project

- Internet Dependency: The web application requires network connectivity for access to the application and its connected services.
- Data Accuracy: Incorrect or incomplete financial entries may affect reports, forecasts, and AIgenerated insights.
- Forecasting Limitations: Cash-flow forecasts depend on available records and assumptions; actual financial outcomes may differ.
- AI Limitations: AI-generated insights may not always be accurate and should be reviewed before making important financial decisions.
- Email Configuration: Email notifications depend on valid email service configuration and successful message delivery.
- Security Risks: Financial information requires appropriate security practices, access control, and regular maintenance.
- Technical Knowledge: Users need basic computer knowledge to operate the application effectively.
- Maintenance and Backups: Regular software updates, database backups, and monitoring are required for reliable operation.

CHAPTER 2: LITERATURE REVIEW

1.1 Overview of Existing Systems

Businesses commonly use spreadsheets, manual registers, accounting software, or separate applications to manage income, expenses, invoices, payments, and financial reports. Although these methods help maintain financial records, using multiple tools can make it difficult to obtain a complete view of the business's financial position. Manual calculations may also lead to errors, delayed payment follow-ups, and difficulties in estimating future cash requirements.

Some existing financial management applications provide digital accounting, expense tracking, invoicing, and reporting features. However, the usefulness of these systems depends on their available features, integrations, and configuration. Businesses may require a combination of financial summaries, payment behavior analysis, cash-flow forecasting, notifications, and AI-assisted insights in one application. The proposed CashFlowGuardian system aims to bring these financial management activities together through a centralized web-based platform.

1.2 Analysis of System 1: Traditional Financial Record Management System

A Traditional Financial Record Management System refers to the use of paper registers, spreadsheets, and basic accounting records to maintain business financial information. Businesses may use these methods to record income, expenses, customer payments, invoices, and other financial transactions. These methods are familiar and can be useful for maintaining basic records, especially when the number of transactions is manageable.

Traditional record management allows users to enter financial information manually and calculate totals using written records or spreadsheet formulas. Users can prepare summaries, track payments, and review expenses by examining the available records. However, the accuracy and usefulness of the information depend heavily on correct data entry, regular updates, and proper organization of the records.

Working of the Existing System

In a traditional system, users record business transactions in registers, spreadsheets, or separate files. Income and expenses are entered manually, while invoices and payments may be maintained in separate records. To understand the financial position, users must review the relevant entries and calculate totals for available cash, outstanding payments, and expenses.

Financial reports are usually prepared by collecting information from these records and performing calculations. When a payment becomes overdue, users may need to check invoice dates manually and contact customers for follow-up. Estimating future cash availability also requires reviewing expected payments and upcoming expenses.

As the volume of transactions increases, maintaining accurate and updated records can become more difficult. Separate files may not provide a unified view of customer payments, outstanding receivables, expenses, and cash-flow trends. These challenges create a need for a centralized financial management application.

Strengths

Traditional financial record management offers several benefits for maintaining basic business records.

Simple Record-Keeping: Users can record income and expenses using registers or spreadsheets without requiring a specialized application.

Low Initial Cost: Basic record management can be performed using existing stationery or spreadsheet software.

Flexible Data Entry: Users can organize records according to their individual financial tracking requirements.

Basic Financial Tracking: Properly maintained records can help users review income, expenses, and payment history.

Easy Familiarity: Users who already work with registers or spreadsheets may find these methods easy to understand.

Customizable Reports: Users can prepare basic financial summaries and reports according to their requirements.

Limitations

Although traditional record management is useful for basic financial tracking, it has several limitations.

Manual Calculations: Repeated calculations can increase the risk of errors and inconsistent financial totals.

Scattered Information: Customer details, invoices, expenses, and payment records may be maintained in different files.

Delayed Payment Follow-Ups: Users may need to check due dates manually to identify overdue invoices.

Limited Forecasting: Estimating future cash availability requires manual analysis of expected receipts and expenses.

Limited Automation: Automatic notifications, scheduled reminders, and AI-assisted financial insights are generally unavailable without additional tools.

Difficult Financial Monitoring: Reviewing large amounts of financial information can take time and make it difficult to identify potential cash shortages.

Data Protection Challenges: Paper records and unprotected spreadsheets may not provide suitable access control or centralized security.

Comparison with the Proposed System

- The proposed CashFlow-Guardian system is designed to address the limitations of traditional financial record management by providing a centralized web-based application. It combines customer management, cash account management, income and expense tracking, invoice management, payment monitoring, and financial summaries in one platform.
- The system uses Java and Spring Boot for backend processing, React.js and TypeScript for the frontend, and PostgreSQL for database management. It also uses Spring Security for authentication and authorization and Spring AI with Google Gemini for AI-assisted financial insights. The dashboard is designed to present relevant financial information through summaries, graphs, and cash-flow indicators.

- Unlike manual record management, CashFlow-Guardian can calculate financial summaries from recorded data, track overdue invoices, generate notifications, and estimate future cash positions. It is intended to reduce manual effort, improve financial visibility, and support better decisionmaking. The accuracy of its summaries and forecasts depends on the quality and completeness of the financial records entered into the system.

1.3 Analysis of System 3: Cloud-Based Accounting and Financial Management System

A Cloud-Based Accounting and Financial Management System is a software solution that enables users to maintain and access financial records through an online application. Such systems may provide features for recording transactions, managing expenses, preparing invoices, tracking payments, and generating financial reports. Depending on the software and subscription plan, they may also support collaboration, automated reminders, and financial dashboards.

These systems can help businesses organize financial information and access records from different devices. By maintaining information in a centralized database, they can reduce the need to manage separate local files and support more consistent financial record-keeping. The actual capabilities depend on the selected application, its configuration, and the services included.

Working of the Existing System

A cloud-based financial management system generally allows authorized users to sign in through a web browser or supported application. After authentication, users can access the financial features available to their accounts. These may include recording income and expenses, maintaining customer information, preparing invoices, recording payments, and reviewing financial summaries.

When users record a financial transaction, the application stores the information in its database and updates the relevant summaries according to the system's design. Users can review outstanding invoices, examine expenses, and generate reports for selected periods. Some systems also provide notifications or integrations to help users manage payment follow-ups.

Cloud-based systems can improve accessibility and organization, but their available financial analysis, forecasting, and AI capabilities vary. Businesses that require

specific workflows may need additional configuration, integrations, or a customized application to meet their requirements.

Strengths

Cloud-based accounting and financial management systems provide several benefits.

Centralized Financial Records: They help maintain financial information in an organized digital environment.

Online Accessibility: Authorized users can access supported features through compatible devices and an internet connection.

Automated Calculations: Supported transaction processing can reduce repeated manual calculations.

Financial Reporting: Reports and summaries help users review income, expenses, and other financial information.

Invoice Management: Available invoicing features can help organize customer invoices and monitor payment statuses.

Improved Organization: Centralized records make it easier to search, review, and maintain financial information.

Integration Opportunities: Some systems support connections with email services, payment providers, or other business applications.

Limitations

The limitations of cloud-based financial management systems depend on their features and configuration.

Feature Availability: Advanced forecasting and AI-assisted analysis may not be available in every application or subscription plan.

Customization Requirements: Business-specific financial workflows may require additional configuration or software development.

Internet Dependency: Access to cloud-hosted services generally requires an internet connection.

Subscription Costs: Some applications require recurring payments for advanced features or additional users.

Integration Constraints: Connecting separate accounting, invoicing, and notification systems may require supported integrations.

Data Quality: Financial reports and forecasts may be inaccurate when the underlying records are incomplete or incorrect.

Security and Privacy: Appropriate authentication, permissions, and data protection measures are essential when handling financial information.

1.4 Summary and Gap Analysis

The analysis of existing financial management systems, traditional accounting methods, and cloud-based financial applications shows that these approaches provide useful features for recording transactions, managing invoices, tracking expenses, and generating financial reports. However, depending on the system, limitations may exist in integrating cash-flow forecasting, customer payment behavior analysis, automated notifications, and AI-assisted financial insights. Managing these activities through separate tools can increase manual effort and make it difficult to obtain a complete view of business finances.

Identified Gaps

- Limited integration of financial records, invoices, expenses, and payments.
- Difficulty monitoring outstanding receivables and overdue invoices.
- Challenges in estimating future cash availability and upcoming obligations.
- Limited integration of customer payment behavior analysis.
- Lack of unified financial dashboards and graphical summaries.
- Limited availability of AI-assisted financial insights and recommendations.

Gap Conclusion

The proposed CashFlow-Guardian system addresses these gaps by providing centralized financial record management, invoice tracking, expense monitoring, cash-flow forecasting, customer payment behavior analysis, and financial notifications through a web-based application. It integrates important financial activities into a single platform, allowing authorized users to review financial information and monitor their business cash position more efficiently.

The system aims to reduce manual work, improve financial visibility, identify potential cash shortages, and support better decision-making. By using React.js, TypeScript, Tailwind CSS, Java, Spring Boot, and PostgreSQL, it provides an organized financial management solution with interactive dashboards, graphical reports, and AI-assisted insights through Spring AI and Google Gemini.

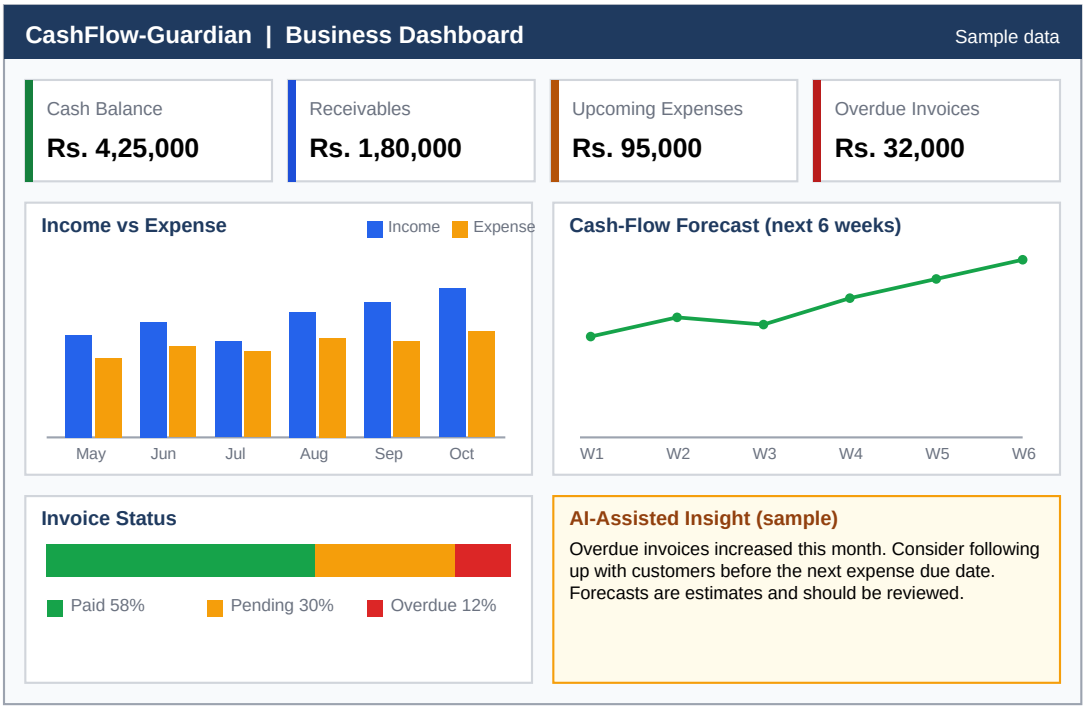


Figure: Illustrative CashFlow-Guardian Dashboard (sample data)

CHAPTER 3: METHODOLOGY

The methodology describes the systematic approach used to develop CashFlow-Guardian: AI-Powered Financial Management System for Business. It includes requirement gathering, system analysis, database design, application development, implementation, and testing. The system is designed to organize financial records, manage invoices and payments, monitor expenses, estimate future cash availability, and provide useful financial insights. It aims to reduce manual effort, improve data accuracy, and support efficient business financial management.

1.1 System Analysis

System analysis is an important phase in the development of CashFlow-Guardian. It involves studying existing financial management methods and identifying the requirements for managing income, expenses, invoices, payments, and cash flow. Traditional methods often involve manual record-keeping, separate spreadsheets, and time-consuming calculations, which may lead to errors and difficulties in monitoring outstanding payments and future financial obligations.

The proposed system provides a centralized web-based solution to overcome these limitations. It allows authorized users to manage customer records, cash accounts, expenses, invoices, payments, and financial summaries. It also supports customer payment behavior analysis, scheduled notifications, cash-flow estimates, and AI-assisted insights. The frontend uses React.js, TypeScript, and Tailwind CSS, while Java and Spring Boot handle backend processing and PostgreSQL stores financial records.

The system analysis focuses on functional requirements, non-functional requirements, user access, database structure, security, and system performance. The application is designed to provide a user-friendly interface, secure access, reliable financial calculations, and organized data management. This analysis establishes the foundation for system design and implementation, ensuring that the proposed solution supports better financial monitoring and informed business decisions.

1.1.1 User Requirements

- User Login: Provides secure access to authorized users.
- User Management: Supports user accounts and appropriate access permissions.

- Business Management: Organizes financial information associated with the business.
- Customer Management: Maintains customer details and related financial records.
- Cash Account Management: Records cash balances and monitors available funds.
- Income and Expense Management: Maintains financial transactions and expense records.
- Invoice Management: Creates and tracks invoices and outstanding amounts.
- Payment Management: Records payments and monitors pending and completed transactions.
- Customer Payment Behavior: Reviews available payment history and identifies payment patterns.
- Cash-Flow Forecasting: Estimates future cash availability using recorded financial information and upcoming obligations.
- Dashboard: Displays important financial summaries, cash balances, receivables, expenses, and cashflow indicators.
- Graphical Reports: Presents financial information through charts and graphical summaries.
- Expense Monitoring: Tracks upcoming and overdue financial obligations.
- Notifications: Generates reminders for due invoices, overdue payments, expenses, and potential cashflow risks.
- AI-Assisted Insights: Uses available financial information to provide AI-assisted explanations and suggestions.
- Search and Filtering: Helps users locate supported financial records efficiently.
- Financial History: Maintains transaction and payment records for review.
- Data Security: Protects financial information using authentication and authorization.
- Data Validation: Validates inputs to reduce errors in financial records.

- User-Friendly Interface: Provides a responsive interface for navigating financial features.

1.1.2 Functional Requirements

- Secure login and authorized access to financial features.
- Customer, cash account, expense, invoice, and payment management.
- Financial summaries and interactive dashboard graphs.
- Cash-flow estimation based on recorded financial information.
- Customer payment behavior analysis and overdue payment tracking.
- Notifications for due invoices, overdue expenses, and cash-pressure alerts.
- AI-assisted financial insights using Spring AI and Google Gemini.
- Search, filtering, validation, and organized financial records.

3.1 SYSTEM DESIGN

The system follows a three-tier architecture:

1. Presentation Layer (Frontend): Designed using React.js, TypeScript, Tailwind CSS, HTML, and CSS to provide an interactive financial dashboard, forms, graphs, and reports.

2. Application Layer (Backend): Uses Java and Spring Boot to handle authentication, business logic, financial calculations, customer records, invoices, expenses, payments, notifications, and REST APIs. Spring Security supports authentication and authorization.

3. Database Layer: Uses PostgreSQL to store business records, customer details, cash accounts, expenses, invoices, payments, notifications, and other supported financial information.

2. Module Design

- User Module: Login, account access, and authorized feature access.
- Customer Module: Maintain customer details and financial relationships.
- Cash Account Module: Record cash balances and monitor available funds.

- Expense Module: Record expenses and track upcoming or overdue obligations.
- Invoice and Payment Module: Manage invoices, payments, and outstanding receivables.
- Behavior Intelligence Module: Analyze customer payment history and identify payment patterns.
- Cash-Flow Module: Calculate financial summaries and estimate future cash positions.
- Notification Module: Generate invoice, expense, and cash-pressure notifications.
- AI Module: Provide AI-assisted financial explanations and suggestions.
- Reports and Dashboard Module: Display financial summaries, graphical trends, and key financial indicators.

3. System Architecture

The system follows a client-server architecture with separate frontend, backend, and database layers •

- Frontend: React.js, TypeScript, and Tailwind CSS provide an interactive and responsive user interface with dashboards, forms, graphs, and reports.
- Backend: Java and Spring Boot process requests, validate inputs, enforce access control, calculate financial summaries, and manage business operations through REST APIs.
- Database: PostgreSQL stores and retrieves customer, cash account, expense, invoice, payment, and notification records.
- AI Integration: Spring AI and Google Gemini support AI-assisted financial insights based on available financial context.
- Security: Spring Security protects supported endpoints through authentication and authorization.
- Development and Testing Tools: IntelliJ IDEA, Visual Studio Code, Git, GitHub, and Postman support development, version control, and API testing.

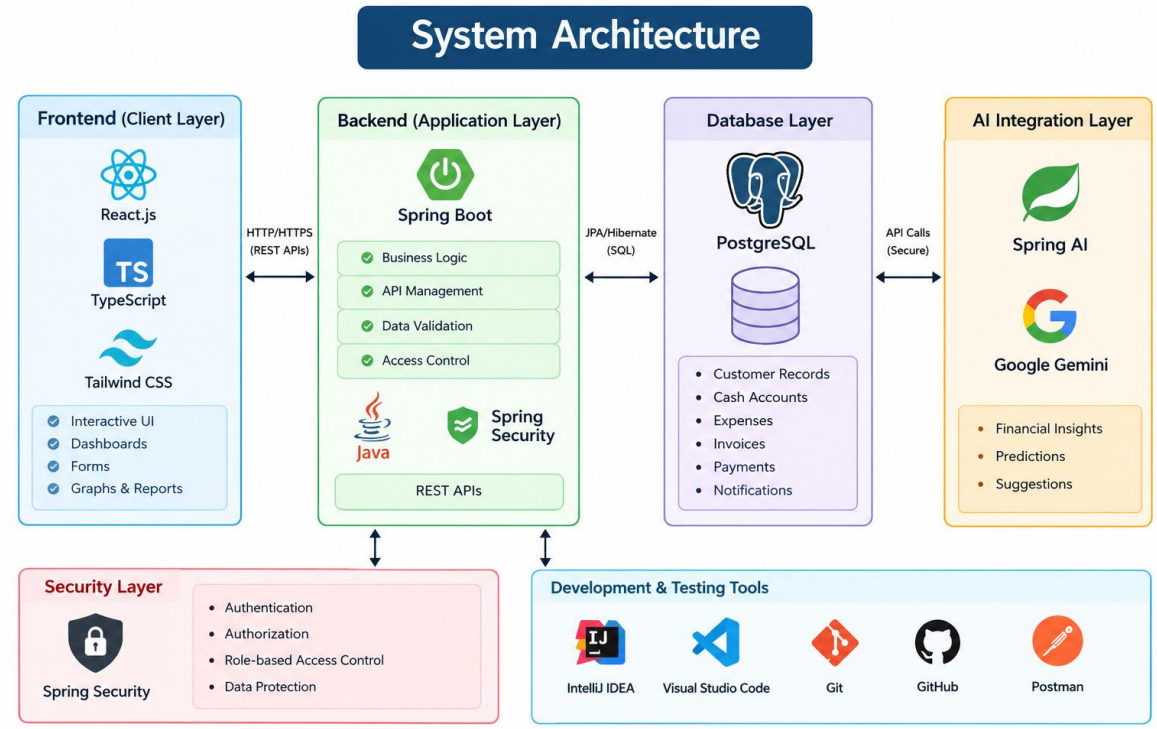


Table 3.1: Core Data Entities of CashFlow-Guardian

Entity	Information described in this report	Related to
User	Login credentials and role-based access to protected features	—
Business	Business-related financial information	—
Customer	Customer details and related business records	Invoice, Payment
Cash Account	Business cash accounts and cash balances	—
Expense	Expense details, amounts, dates, categories; upcoming or overdue obligations	—
Invoice	Customer invoice records with pending, due or overdue status	Customer, Payment
Payment	Payment records associated with transactions and invoices	Invoice, Customer
Notification	Invoice and expense due dates, overdue payments, cash-pressure alerts	—

CHAPTER 4: PROPOSED SYSTEM AND IMPLEMENTATION

1.1 Working of Proposed System

The proposed CashFlow-Guardian: AI-Powered Financial Management System for Business is a web-based application designed to simplify and improve business financial management. The system provides a centralized platform for managing customers, cash accounts, income, expenses, invoices, payments, notifications, and financial reports. It helps authorized users maintain organized financial records and monitor important financial activities through an interactive dashboard. The application uses React.js, TypeScript, and Tailwind CSS for the frontend, Java and Spring Boot for backend processing, and PostgreSQL for database management. Spring AI and Google Gemini are used to support AI-assisted financial insights based on available financial information.

1. User Authentication and Login: The system provides a secure login mechanism for authorized users. Users enter their login credentials to access the application. Spring Security supports authentication and authorization for protected endpoints. Access to supported operations is controlled according to the application's security configuration. This helps protect business financial information and prevents unauthorized access.

2. Dashboard Management: After successful login, users can access the main dashboard to view important financial information in an organized format. The dashboard is designed to present financial summaries, income and expense information, cash account details, invoice status, payment information, and important notifications. Graphs and charts help users understand financial activities more easily. The dashboard provides quick access to important modules and supports better financial monitoring

3. Customer Management: The customer management module maintains customer information and related business records. Authorized users can manage customer details and access relevant information associated with invoices and payments. Organized customer records make it easier to track financial transactions, review outstanding payments, and maintain consistent business information.

4. Cash Account Management: The cash account module helps maintain records of business cash accounts and their associated financial information. It supports the organization of account-related records and helps users monitor available cash information. By maintaining centralized cash account records, the system makes financial information easier to access and review.

5. Expense Management: The expense management module records and organizes business expenses. Users can maintain expense details, amounts, dates, categories, and other supported information. The system helps users review recorded expenses and understand where business funds are being used. Organized expense records support financial monitoring, expense analysis, and cash-flow planning

6. Income and Transaction Management: The system maintains financial transaction records associated with business activities. Income and other supported financial entries can be recorded and reviewed to provide a clearer view of money received and money spent. These records help calculate financial summaries and support the monitoring of cash movement over time. Maintaining organized transaction information also improves the availability of data for reports and analysis.

7. Invoice Management: The invoice management module helps maintain customer invoice records and their payment status. Authorized users can manage supported invoice information and review invoices that are pending, due, or overdue. Organized invoice records help users identify amounts that are expected to be received and monitor upcoming payment obligations. This module supports better tracking of business receivables.

8. Payment Management: The payment management module maintains payment records associated with relevant financial transactions and invoices. It helps users review payment information and monitor the status of recorded payments. By connecting payment records with relevant business information, the system supports more organized financial tracking and helps users understand outstanding amounts. Accurate payment records also contribute to financial summaries and customer payment-behavior analysis.

9. Payment Behavior Intelligence: The Behavior Intelligence module analyzes available customer payment information to help identify payment patterns. It uses recorded payment-related information to support the evaluation of customer payment behavior. These insights can help businesses understand payment

regularity and identify customers whose payments may require closer monitoring. The module is intended to support better payment follow-up and financial planning based on available records.

10. Cash-Flow Monitoring and Forecasting: The cash-flow module helps users understand the movement of money within the business by considering available financial records. It supports financial summaries and is designed to estimate future cash availability using recorded transactions and relevant upcoming financial obligations. Forecasting can help identify possible cash shortages in advance and support better planning of business expenses and expected receipts. Forecast values are estimates based on available information and may differ from actual future results.

11. Notifications and Financial Alerts: The notification module helps users monitor important financial events, including upcoming invoice due dates, overdue invoices, expense due dates, and possible cash-flow pressure. The system can generate relevant notifications based on available records and configured rules. These alerts help users pay attention to financial obligations and take timely action. Notification availability depends on the application's configured conditions and supported features.

12. AI-Powered Financial Insights: CashFlow-Guardian integrates Spring AI with Google Gemini to support AI-assisted financial analysis. The AI integration is intended to interpret relevant financial context and provide understandable insights, suggestions, and observations about business financial activities. It can help users review spending patterns and understand potential financial concerns. AIgenerated results are intended to support decision-making and should be reviewed alongside actual financial records rather than treated as guaranteed predictions.

13. Reports and Analytics: The system organizes financial information to support business monitoring and analysis. Financial summaries and graphical representations can help users review income, expenses, cash accounts, invoices, payments, and other available financial records. These reports and visual indicators make it easier to understand financial activity and identify areas that may need attention. The scope of individual reports depends on the information and reporting features available in the application.

14. Database Management: PostgreSQL is used to store and manage application information, including customer records, cash account details, expenses, invoices,

payments, and notification records. Java and Spring Boot communicate with the database through the application's data-access components, including Spring Data JPA and Hibernate. Appropriate entity relationships, validation, and database operations help maintain organized and consistent records. The database acts as a centralized source of information for the application's supported modules.

15. Security and Access Control: The system uses Spring Security to support authentication and authorization for protected application endpoints. Input validation and exception handling help manage invalid requests and reduce unexpected application errors. Sensitive configuration values, such as API keys and database credentials, should be maintained outside public source code and protected through environment-based configuration. Appropriate access controls and secure development practices help protect business information.

16. Centralized Financial Monitoring: CashFlow-Guardian provides a centralized platform for reviewing supported financial records instead of depending entirely on separate spreadsheets or manual records. Users can access relevant customer, expense, invoice, payment, and cash-flow information through the application's modules and dashboard. This centralized approach helps reduce information fragmentation and makes important financial details easier to review. It also supports more organized financial planning and follow-up activities.

17. Implementation and Deployment: The frontend is developed using React.js, TypeScript, and Tailwind CSS, while Java and Spring Boot handle backend operations through REST APIs. PostgreSQL stores the application's financial records. Spring Security supports protected operations, and Spring AI with Google Gemini provides the AI integration. IntelliJ IDEA, Visual Studio Code, Git, GitHub, and Postman support development, version control, and API testing. The application can be deployed in a suitable environment after the required configuration, testing, security checks, and financial insights into a single platform. The proposed CashFlow-Guardian system aims to reduce manual effort, organize financial data, provide timely notifications, and help users identify potential financial risks. By presenting relevant financial summaries and estimates based on recorded data, the system supports better planning, improved financial visibility, and informed business decisions.

1.2 Hardware and Software

1.2.1 Hardware Requirement

- Processor: Intel Core i3 or higher, or an equivalent processor.
- RAM: Minimum 4 GB; 8 GB or more recommended for development.
- Hard Disk: Minimum 10 GB of free space for project files and development tools, with additional space for databases and dependencies.
- Monitor: Standard LCD/LED monitor.
- Keyboard and Mouse: Standard input devices.
- Internet Connection: Required for downloading dependencies, accessing online AI services, and other internet-dependent development or application features.

1.2.2 Software Requirement

- Operating System: Windows 10/11 or another compatible operating system.
- Frontend: React.js, TypeScript, Tailwind CSS, HTML, and CSS.
- Backend: Java and Spring Boot.
- Database: PostgreSQL.
- Data Access: Spring Data JPA and Hibernate.
- Security: Spring Security.
- AI Integration: Spring AI and Google Gemini.
- Development Tools: IntelliJ IDEA and Visual Studio Code.
- Version Control: Git and GitHub.
- API Testing Tool: Postman.
- Web Browser: Google Chrome or Microsoft Edge.

1.3 Testing and Implementation

Testing and implementation ensure that CashFlow-Guardian functions according to its requirements and handles financial information consistently. Testing helps identify errors, verify the behavior of individual modules, and check communication between the frontend, backend, and database. The following testing methods are relevant to evaluating the system.

- Unit Testing: Individual components and methods, such as financial calculations, validation rules, and business services, are tested separately to verify their behavior.
- Integration Testing: The interaction between controllers, services, repositories, and PostgreSQL is tested to ensure that data is processed and stored correctly.
- System Testing: The complete application is evaluated to verify that its supported modules work together according to the defined requirements.
- User Acceptance Testing (UAT): The application can be evaluated by intended users to determine whether its interface, financial information, and workflows meet their needs.
- Security Testing: Authentication, authorization, protected endpoints, configuration security, and prevention of unauthorized access are reviewed.
- Performance Testing: Response times and application stability are evaluated during operations such as loading financial summaries, retrieving records, and processing requests.
- Database Testing: PostgreSQL connectivity, record creation, updating, retrieval, relationships, and data consistency are verified.
- Functional Testing: Supported features, including customer management, cash accounts, expense management, invoices, payments, and notifications, are checked against their requirements.
- Validation Testing: User inputs, financial amounts, required fields, and relevant transaction details are checked before processing or saving records.
- Compatibility Testing: The frontend is checked in supported web browsers and across different screen sizes to verify that the interface remains usable.
- Error Handling Testing: The application is checked to ensure that invalid requests and expected application errors are handled appropriately.

- Financial Calculation Testing: Financial summaries and cash-flow calculations are checked against sample records to verify the correctness of cal
- Notification Testing: Notification conditions for supported invoice and expense due dates, overdue
- AI Integration Testing: The Spring AI and Google Gemini integration is checked for successful requests, appropriate handling of responses, and errors such as unavailable services or invalid configuration.
- Report Testing: Available financial summaries, graphical information, and reports are checked for consistency with the underlying records.
- Backup and Recovery Testing: Database backup and restoration procedures can be tested to help ensure that business records can be recovered when required.
- Implementation: The React frontend, Spring Boot backend, PostgreSQL database, security configuration, and AI integration are configured to work together in the selected development or deployment environment.
- Final Verification: The complete application is reviewed to confirm that its required features, database connectivity, validation, security configuration, and error handling work as expected before deployment.

```
@Service
public class CashFlowService {

    private final CashAccountRepository cashAccountRepository;
    private final InvoiceRepository invoiceRepository;
    private final ExpenseRepository expenseRepository;

    public CashFlowService(CashAccountRepository cashAccountRepository,
                           InvoiceRepository invoiceRepository,
                           ExpenseRepository expenseRepository) {
        this.cashAccountRepository = cashAccountRepository;
        this.invoiceRepository = invoiceRepository;
        this.expenseRepository = expenseRepository;
    }



    // Estimated cash = balance + expected receipts - upcoming expenses
    public BigDecimal forecastCashPosition(Long businessId, LocalDate untilDate) {
        BigDecimal balance = cashAccountRepository.sumBalance(businessId);
        BigDecimal receivables =
                invoiceRepository.sumOutstandingDueBy(businessId, untilDate);
        BigDecimal expenses =
                expenseRepository.sumUnpaidDueBy(businessId, untilDate);
        return balance.add(receivables).subtract(expenses);
    }
}
```

CHAPTER 5: CONCLUSION

5.1 Conclusion

The CashFlow-Guardian: AI-Powered Financial Management System for Business is designed to provide a centralized and organized solution for managing business financial information. The system brings together important activities such as customer management, cash account management, expense tracking, invoice management, payment monitoring, and financial summaries. By organizing these activities within a single application, it aims to reduce dependence on scattered manual records and improve access to important financial information.

The proposed system includes features for maintaining customer records, recording expenses, managing cash accounts, tracking invoices, and reviewing payment information. These features help users organize financial activities and understand outstanding payment obligations. The dashboard and graphical financial information are intended to make important financial details easier to understand and support more effective monitoring of business finances.

CashFlow-Guardian also incorporates payment behavior analysis and cash-flow monitoring. By using available financial records, the system can help users review customer payment patterns and estimate future cash availability. Notifications for supported financial events can help users pay attention to upcoming obligations, overdue invoices, and possible cash-flow pressure. These features aim to support timely financial planning, although estimates and alerts depend on the accuracy and completeness of the available records.

The application uses React.js, TypeScript, and Tailwind CSS for the frontend, Java and Spring Boot for backend processing, and PostgreSQL for database management. Spring Data JPA and Hibernate support database operations, while Spring Security helps protect supported endpoints. Spring AI and Google Gemini provide an integration for AI-assisted financial insights. This technology stack provides a structured foundation for developing a maintainable web-based financial management application.

Overall, CashFlow-Guardian aims to improve the organization, accessibility, and monitoring of business financial information. Its combination of financial record management, payment tracking, cash-flow estimates, notifications, and AI-assisted analysis provides a practical foundation for supporting business financial decisions.

The usefulness of the system depends on accurate data, appropriate configuration, testing, and responsible interpretation of AI-generated insights. Further development and evaluation can enhance its capabilities and suitability for different business requirements.

5.2 Future Scope

- Mobile Application: Develop a mobile application to allow authorized users to access financial summaries, review invoices, and monitor important notifications from mobile devices.
- Cloud Integration: Deploy the application in a suitable cloud environment to improve accessibility, availability, and infrastructure scalability.
- Advanced Cash-Flow Forecasting: Improve forecasting by considering historical transactions, upcoming expenses, invoice due dates, payment patterns, and additional relevant financial data.
- Automated Financial Alerts: Extend notification capabilities with configurable alerts for upcoming obligations, overdue invoices, unusual financial changes, and potential cash shortages.
- Advanced Reports and Analytics: Introduce more detailed financial reports, customizable date ranges, graphical comparisons, and downloadable reports for business analysis.
- Payment Gateway Integration: Integrate suitable authorized payment services to support payment workflows where required, subject to provider availability and applicable security requirements.
- AI-Powered Financial Assistant: Extend the Spring AI and Google Gemini integration to support natural-language questions about available financial records and provide contextual explanations of financial summaries.
- Financial Anomaly Detection: Develop additional analytical features to identify unusual expenses, unexpected transaction patterns, and inconsistencies that may require further review.
- Customer Payment Risk Analysis: Improve payment behavior analysis using additional historical data and suitable analytical methods to help identify potential payment delays.

- Data Security and Audit Trail: Strengthen security through appropriate access controls, secure configuration, audit records, and monitoring of important financial operations.
- Export and Integration Features: Add options to export supported reports and integrate with suitable accounting or business applications through secure APIs.
- Scalability and Performance Optimization: Improve database queries, caching where appropriate, monitoring, and deployment configuration to support increased data volumes and application usage.

CHAPTER 6: REFERENCES

- React Documentation. Available at: https://react.dev/
- TypeScript Documentation. Available at: https://www.typescriptlang.org/docs/
- Tailwind CSS Documentation. Available at: https://tailwindcss.com/docs
- Java Documentation. Available at: https://docs.oracle.com/en/java/
- Spring Boot Documentation. Available at: https://docs.spring.io/spring-boot/
- Spring Data JPA Documentation. Available at: https://docs.spring.io/spring-data/jpa/reference/
- Hibernate ORM Documentation. Available at: https://hibernate.org/orm/documentation/
- Spring Security Documentation. Available at: https://docs.spring.io/spring-security/reference/
- Spring AI Documentation. Available at: https://docs.spring.io/spring-ai/reference/
- Google Gemini API Documentation. Available at: https://ai.google.dev/gemini-api/docs
- PostgreSQL Documentation. Available at: https://www.postgresql.org/docs/
- Git Documentation. Available at: https://git-scm.com/doc
- GitHub Documentation. Available at: https://docs.github.com/
- Postman Documentation. Available at: https://learning.postman.com/docs/
- MDN Web Docs. Available at: https://developer.mozilla.org/